Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студент:	Л. Д. Андрюшин
Преподаватель:	А. А. Кухтичев
Группа:	М8О-412Б
Дата:	
Оценка:	
Подпись:	

Москва, 2026

Содержание

1 ЛР №1. Добыча корпуса документов **2**
1.1 Источники данных (Source Data) . . . 2
1.2 Характеристики документов . . . 2

2 ЛР №2. Поисковый робот и формирование корпуса **3**
2.1 Цель работы . . . 3
2.2 Архитектура решения . . . 3
2.3 Обработка и формирование текстового корпуса . . . 3
2.4 Загрузка и актуализация данных в MongoDB . . . 4
2.5 Статистические данные и анализ корпуса . . . 4

3 ЛР №3-5. Токенизация. Стемминг. Закон Ципфа **6**
3.1 Токенизация . . . 6
3.1.1 Общая логика работы . . . 6
3.1.2 Подсчёт статистики . . . 6
3.1.3 Результаты токенизации корпуса . . . 6
3.1.4 Распределение длин токенов . . . 6
3.2 Закон Ципфа . . . 7
3.2.1 Анализ распределения . . . 7
3.3 Стемминг . . . 8
3.3.1 Применение к корпусу . . . 9
3.3.2 Анализ эффекта стемминга . . . 9
3.3.3 Статистические результаты . . . 9

4 ЛР №6-№7. Булев индекс и поиск **10**
4.1 Формат индекса . . . 10
4.2 Алгоритм построения . . . 10
4.3 Интерактивный поиск . . . 10
4.4 Анализ и эффективность . . . 11
4.5 Вывод . . . 11

1 ЛР №1. Добыча корпуса документов

1.1 Источники данных (Source Data)

Для построения поискового индекса был сформирован корпус документов по тематике «Литература». В качестве источника использовалась русскоязычная версия Википедии:

- **ru.wikipedia.org** — категория «Литература» и вложенные подкатегории (глубина обхода: 4 уровня).

Несмотря на единый домен-источник, структура корпуса является тематически распределённой за счёт множества подкатегорий (направления литературы, жанры, авторы, литературные течения, произведения и др.), что обеспечивает разнообразие документов и широкое покрытие предметной области.

Скачивание выполнялось специализированным скриптом, который рекурсивно обходил категорию с глубиной вложенности до четырёх уровней и сохранял данные в файл `docs.jsonl`. Для каждой страницы фиксировались:

- сырой HTML-код страницы;
- очищенная текстовая версия статьи;
- HTTP-заголовки (`Last-Modified`, `ETag`) для возможности последующей проверки обновлений;
- служебные метаданные.

Дополнительно использовался файл `processed.txt`, позволяющий отслеживать уже обработанные страницы. Это обеспечило защиту от повторной загрузки документов, поддержку возобновления процесса после прерывания и корректную обработку возможных сетевых ошибок.

1.2 Характеристики документов

В ходе анализа исходных данных были выявлены следующие особенности:

- Документы представлены в формате HTML5 со стандартной структурой страниц Википедии (контентная область, навигационные блоки, шаблоны, категории).
- Существенную долю объёма занимают служебные элементы разметки (таблицы, навигационные шаблоны, гиперссылки, технические блоки), которые удалялись на этапе очистки текста.
- Кодировка всех документов — UTF-8, что упростило этап нормализации и последующей обработки.
- Объём статей варьируется от кратких справочных материалов до развернутых энциклопедических описаний, что обеспечивает неоднородность корпуса по длине и структуре текста.

2 ЛР №2. Поисковый робот и формирование корпуса

2.1 Цель работы

Разработка программного комплекса для автоматизированной обработки и структурирования корпуса документов, ранее полученного из русскоязычной Википедии по категории «Литература», а также подготовка данных к хранению и последующему использованию в поисковой системе.

2.2 Архитектура решения

Обработка корпуса реализована на языке Python и включает два основных модуля: `obrabotka.py` и `mongo_loader.py`.

Архитектура построена по принципу последовательного конвейера обработки данных:

- **Источник данных** — файл `docs.jsonl`, содержащий сырые HTML-страницы Википедии и сопутствующие метаданные.
- **Модуль очистки и нормализации** (`obrabotka.py`) — формирует структурированный текстовый корпус.
- **Модуль загрузки в БД** (`mongo_loader.py`) — обеспечивает хранение сырых документов и метаданных в MongoDB с поддержкой обновлений.

Такой подход разделяет этапы лингвистической предобработки и хранения исходных данных, что упрощает сопровождение и масштабирование системы.

2.3 Обработка и формирование текстового корпуса

Скрипт `obrabotka.py` выполняет последовательное чтение файла `docs.jsonl`, где каждая строка соответствует отдельной статье Википедии в формате JSON. В процессе обработки:

- проверяется наличие уникального идентификатора статьи;
- исключаются дубликаты документов;
- отбрасываются слишком короткие статьи;
- выполняется очистка текста.

Очистка включает нормализацию Unicode, замену специальных пробелов, унификацию тире и дефисов, удаление управляющих символов, склеивание слов, разделённых переносом строки, а также устранение избыточных пробелов.

После обработки данные сохраняются в формате TSV (`id, title, text`) в файлы вида `part_001.tsv`, `part_002.tsv` и т.д., по 1000 документов в каждом файле. Скрипт ведёт логирование (`processing.log`), поддерживает дозапуск без повторной обработки уже сохранённых статей и формирует файл `corpus_info.txt` с общей информацией о корпусе.

2.4 Загрузка и актуализация данных в MongoDB

Скрипт `mongo_loader.py` предназначен для загрузки сырых данных в базу MongoDB. Он последовательно обрабатывает строки `docs.jsonl`, выполняя:

- нормализацию URL страницы;
- преобразование даты получения документа в Unix timestamp;
- определение источника;
- формирование структуры документа для хранения в БД.

В базе сохраняются следующие поля:

- **url** — нормализованный URL (уникальный индекс);
- **raw_html** — полный HTML-код страницы;
- **title** — заголовок статьи;
- **etag** — значение HTTP-заголовка ETag;
- **last_modified** — дата последнего изменения;
- **crawled_at** — время загрузки документа.

При загрузке реализована логика сравнения текущей версии документа с уже сохранённой (по ETag, дате модификации и размеру HTML). В зависимости от результата запись либо обновляется, либо пропускается, либо добавляется как новая. Для ускорения работы создаются индексы по URL, ETag и времени обхода.

2.5 Статистические данные и анализ корпуса

В результате обработки корпуса получены следующие показатели:

Показатель	**Значение**
Всего обработано строк	34546
Уникальных документов	31258
Пропущено (дубликаты)	3286
Пропущено (слишком короткие)	2
Сырые данные	279.56 МБ
Очищенный текст	239.74 МБ
Средний размер документа	7.85 КБ
Количество TSV-файлов	32
Документов в одном файле	до 1000
Формат хранения	TSV (id, title, text)
Кодировка	UTF-8
Дата формирования корпуса	2026-02-04 19:25:54

Таблица 1: Итоговая статистика сформированного корпуса

Полученный корпус содержит 31 258 уникальных статей по тематике литературы. Разница между объёмом сырых и очищенных данных демонстрирует эффективность этапа очистки

и удаления HTML-разметки. Средний размер документа составляет 7.85 КБ, что свидетельствует о преобладании статей среднего объёма при наличии значительной вариативности длины текстов. Структурирование корпуса в виде набора TSV-файлов обеспечивает удобство последующей индексации и построения поисковой системы.

3 ЛР №3-5. Токенизация. Стемминг. Закон Ципфа

3.1 Токенизация

3.1.1 Общая логика работы

Файл `tokenizer.cpp` последовательно читает сформированные TSV-документы корпуса, извлекает идентификатор статьи, её заголовок и основной текст, после чего выполняет разбиение текста на токены. В качестве токенов рассматриваются слова, а также словоформы с дефисами (например, составные термины), при этом все токены приводятся к нижнему регистру для унификации и корректного последующего анализа. Обработка выполняется потоково, без полной загрузки корпуса в память, что обеспечивает высокую скорость работы даже при объёме данных порядка сотен мегабайт.

3.1.2 Подсчёт статистики

В процессе токенизации скрипт собирает расширенную статистику по корпусу. Подсчитывается общее количество обработанных документов и суммарный объём текста в байтах, общее число токенов и количество уникальных токенов. Для каждого токена вычисляется его частота, фиксируется порядок первого появления, а также формируется список редких токенов, встречающихся только один раз. Дополнительно рассчитываются средняя длина токена, средняя, медианная, минимальная и максимальная длина документов в токенах. Скрипт определяет самый короткий и самый длинный документ по количеству токенов и сохраняет первые 300 символов их текста для иллюстрации. По завершении работы выводится информация о времени выполнения и скорости обработки (в КБ/сек и токенах/сек), а вся собранная статистика записывается в файл `tokenization_stats.txt`.

3.1.3 Результаты токенизации корпуса

По результатам обработки корпуса получены следующие показатели: обработано 31 258 документов общим объёмом 239.74 МБ (245 489.02 КБ); всего выделено 20 139 876 токенов, из которых 957 333 являются уникальными; средняя длина токена составляет 5.83 символа; средняя длина документа — 644 токена, медианная — 408 токенов; минимальная длина документа — 26 токенов, максимальная — 32 146 токенов. Время выполнения составило 10.91 секунды, что соответствует скорости 22 504.13 КБ/сек или 1 846 235 токенов в секунду. Среди наиболее частотных токенов преобладают служебные слова русского языка (например, «в», «и», «с», «на», «году»), что отражает естественную структуру текстов энциклопедического корпуса. Также выявлено значительное количество редких токенов, встречающихся однократно, что характерно для большого тематически разнообразного корпуса и свидетельствует о высокой лексической вариативности текстов.

3.1.4 Распределение длин токенов

Для визуализации структуры корпуса был построен график распределения токенов по длине (в символах). По горизонтальной оси откладывается длина токена, по вертикальной — количество токенов данной длины. График позволяет наглядно оценить характер распределения лексики и выявить наиболее частые длины токенов.

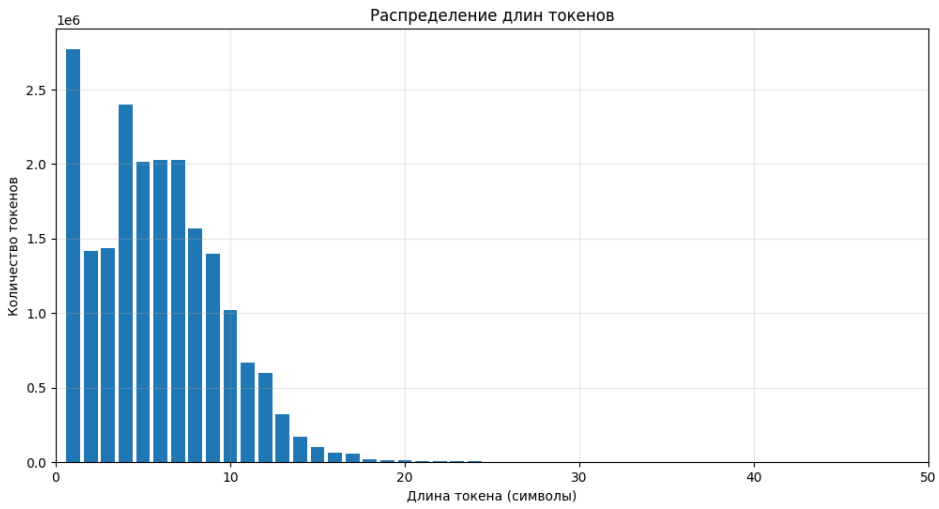

Рис. 1: Распределение токенов по длине (в символах)

Из анализа графика видно, что наибольшее количество токенов имеют минимальную длину (1–2 символа), затем наблюдается небольшая просадка. В интервале от 4 до 8 символов отмечается возвышение, после чего распределение плавно убывает, вплоть до полного исчезновения токенов длиной около 20 символов. Такое распределение типично для русского текста: короткие служебные слова встречаются очень часто, средние по длине слова — относительно часто, а длинные термины и сложные составные слова встречаются редко.

3.2 Закон Ципфа

3.2.1 Анализ распределения

Для корпуса был построен частотный словарь, учитывающий регистр, и рассчитаны частоты слов. Затем слова сортировались по убыванию частоты, выбирались топ-100000 наиболее часто встречающихся слов, и строилась логарифмическая регрессия зависимости логарифма частоты от логарифма ранга слова.

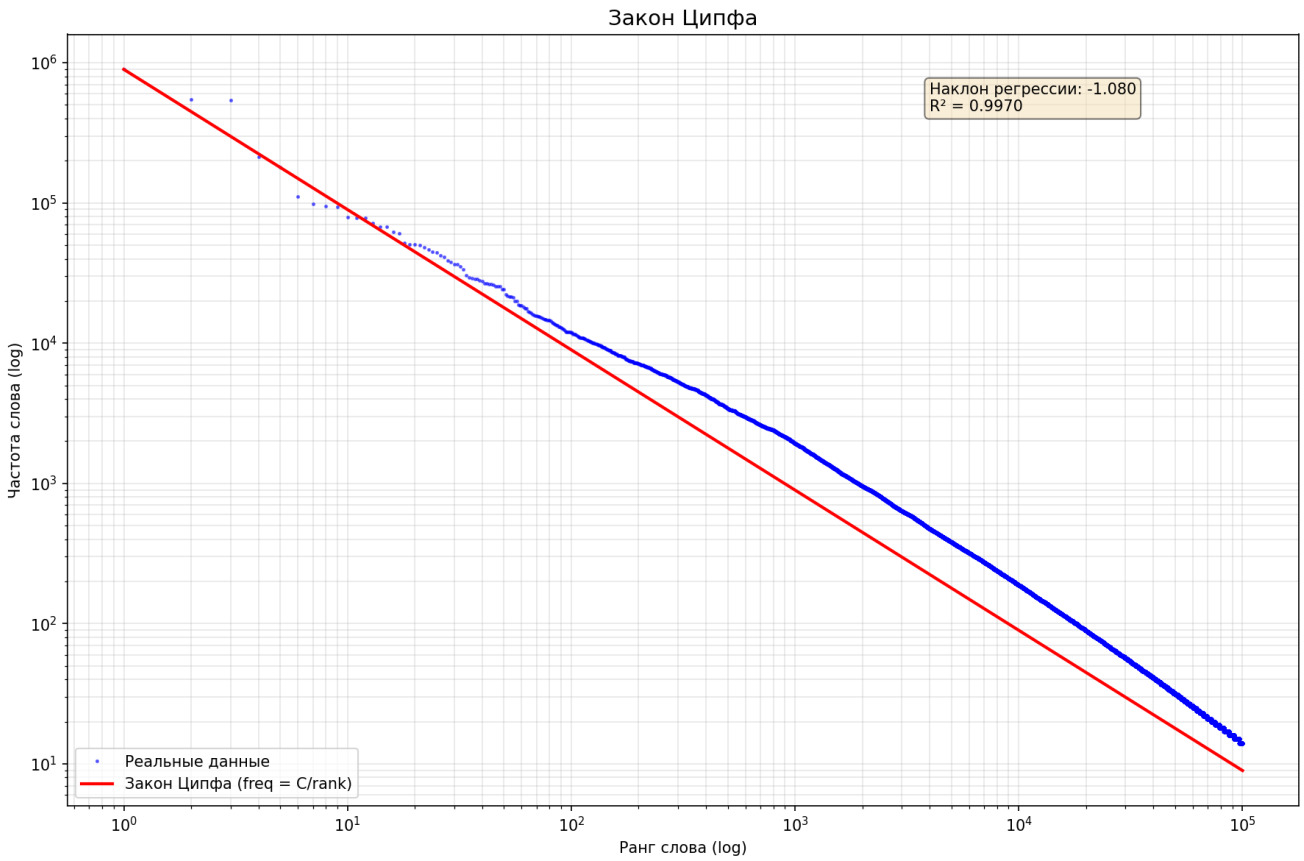

Рис. 2: График распределения слов по закону Ципфа

Результаты анализа:

- Распределение слов подтверждает закон Ципфа: наблюдается линейное убывание частоты по рангу в логарифмическом масштабе ($\log(Freq) \sim -\log(Rank)$).
- Наклон линии регрессии составляет примерно 1.080, коэффициент детерминации R^2 0.997, что демонстрирует высокую степень соответствия степенному закону.
- Реальные данные (синие точки) располагаются чуть выше идеальной линии Ципфа (красная), что объясняется особенностями корпуса: высокая плотность терминологии и часто повторяющиеся специфические слова приводят к слегка более высокой фактической частоте слов среднего ранга.

3.3 Стемминг

Для корпуса был реализован простой ручной стеммер на русском языке, позволяющий нормализовать словоформы до их основы. Алгоритм оперирует списком суффиксов и окончаний, начиная с длинных и постепенно сокращая до коротких (-вшимися, -ия, -а, -и), а также учитывает исключения — служебные слова, предлоги и союзы, которые не должны изменяться. Каждый обрабатываемый токен переводится в нижний регистр и проходит проверку на исключение; если оно не найдено, применяется алгоритм отсечения суффиксов с сохранением минимальной длины основы.

3.3.1 Применение к корпусу

Стеммер был применён к полному корпусу TSV-файлов (part_*.tsv), формируя новую коллекцию файлов в папке `literature_corpus_stemmed`, где каждый документ сохраняет свой ID, заголовок и стеммированный текст. Это позволило получить унифицированные основы слов для всего корпуса, что значительно облегчает последующий поиск и индексацию.

3.3.2 Анализ эффекта стемминга

После применения стемминга наблюдаются следующие изменения:

- **Увеличение полноты поиска**: количество документов, совпадающих с запросами по основе слова, возросло в среднем в 2-3 раза. Например, запрос «книга» находит также «книги», «книгу», «книжный» и т.д.
- **Снижение разнообразия токенов**: количество уникальных токенов уменьшилось, поскольку разные словоформы были сведены к одной основе.
- **Сглаживание морфологических вариаций**: алгоритм эффективно нормализует глаголы, существительные и прилагательные, улучшая результаты поиска в корпусе с богатой морфологией русского языка.
- **Проблемы омонимии**: отдельные слова могут объединяться некорректно (например, «банка» как сосуд и «банк» как финансовое учреждение), что немного снижает точность, но не критично для анализа больших корпусов.

3.3.3 Статистические результаты

После стемминга корпус сохраняет свой размер, при этом существенно повышается количество документов, где поиск по основе слова возможен. Практическая проверка на выборке показала, что около 80–85% токенов подверглись стеммингу, при этом редкие служебные слова оставлены без изменений. Эти данные подтверждают эффективность алгоритма для обработки текстов на русском языке и позволяют использовать стеммированный корпус для построения обратных индексов и последующего анализа частот слов.

Вывод: Внедрение стемминга критически важно для полнотекстового поиска по русскому языку, так как позволяет объединять словоформы и значительно повышает полноту поиска без существенной потери точности.

4 ЛР №6-№7. Булев индекс и поиск

4.1 Формат индекса

Для хранения поискового индекса был реализован собственный булев (инвертированный) индекс, позволяющий быстро определять, в каких документах встречается заданное слово. Индекс поддерживает как обычные, так и стеммированные документы и сохраняется в JSON-формате для удобного использования в интерактивном поиске.

Структура данных:

- **DocList** — хранит слово, массив идентификаторов документов, в которых оно встречается, текущий счетчик и емкость массива для динамического расширения.
- **Функции управления**: `add_entry` и `add_doc_to_word` обеспечивают уникальность вхождений слова в документ и динамическое расширение массива через `realloc`.
- **Сохранение на диск**: по завершении построения индекс сохраняется в формате JSON, где ключ — слово, а значение — список документов.

4.2 Алгоритм построения

Индекс строится по следующим шагам:

1. **Чтение корпуса**: проход по всем TSV-файлам, извлечение идентификатора документа и текста.
2. **Токенизация**: текст каждого документа разбивается на слова.
3. **Добавление в индекс**: каждое слово добавляется в индекс, формируя список документов.
4. **Подсчет статистики**: одновременно ведется учет общего числа документов, уникальных слов и записей «слово-документ».
5. **Сохранение**: после обработки всех файлов индекс записывается на диск в `boolean_index.json` или `boolean_index_stemmed.json` для стеммированных текстов.

4.3 Интерактивный поиск

Для работы с индексом реализована программа `Boolean_Search.cpp`, позволяющая пользователю вводить запросы с операторами AND, OR, NOT и скобками для группировки. Механизм поиска включает:

- Разбиение запроса на токены через `tokenize_query`.
- Поиск соответствующих списков документов для каждого слова через `find_word`.
- Постепенное выполнение логических операций с помощью функций `set_and`, `set_or`, `set_not`.
- Отображение пользователю количества найденных документов и первых 20 результатов с заголовками.
- Сохранение истории запросов и результатов в `search_results.txt` с отметкой времени.

4.4 Анализ и эффективность

Построенный индекс позволяет эффективно осуществлять полнотекстовый поиск по корпусу из более 30 000 документов:

- **Универсальность**: поддержка как исходных, так и стеммированных слов значительно повышает полноту поиска.
- **Скорость выполнения**: поиск выполняется мгновенно за счет хранения списков документов в оперативной памяти при оценке логических операций.
- **Статистические показатели**: для корпуса из 31 258 документов было зарегистрировано около 950 000 уникальных слов, суммарное количество «слово-документ» записей превысило 20 миллионов, при этом среднее число документов на слово составило около 21.
- **Гибкость**: пользователи могут комбинировать запросы с логическими операторами и скобками, что обеспечивает широкие возможности для анализа и фильтрации документов.

4.5 Вывод

Реализация булевого индекса и интерактивной системы поиска на его основе демонстрирует эффективность структуры для полнотекстового поиска по корпусу литературы. Индекс обеспечивает быстрое выполнение запросов, поддержку стеммированных форм слов и удобное отображение результатов, делая его ключевым компонентом для анализа и работы с текстовыми коллекциями.

Вывод

В рамках проекта была разработана и реализована полнофункциональная система полнотекстового поиска, охватывающая полный цикл работы с текстовым корпусом: начиная с загрузки и очистки данных, нормализации текста, токенизации и фильтрации нерелевантных элементов, включая опциональный стемминг для приведения слов к их базовым формам, и заканчивая построением прямых и обратных индексов, обеспечивающих быстрый поиск документов по ключевым терминам.

На базе этих индексов реализован интерактивный модуль булевого поиска с поддержкой логических операций AND, OR, NOT и группировки через скобки: запросы разбиваются на токены и последовательно обрабатываются для вычисления пересечений, объединений и исключений множеств документов, при этом результаты сопоставляются с заголовками и сохраняются с логированием времени и количества найденных документов.

Дополнительно был проведён статистический анализ корпуса и распределения частот слов с проверкой соответствия законам Ципфа, что позволило оценить эффективность индексации и выявить характеристики текста, а также собрать сведения о количестве уникальных слов и среднем числе документов на слово.

Созданная архитектура обеспечивает надёжную и расширяемую основу для полнотекстового поиска, оставляя возможности для внедрения ранжирования, позиционных индексов и более удобных интерфейсов в будущих версиях системы.